

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México



**Momento de Retroalimentación: Implementación de una técnica de
aprendizaje máquina con framework**

Random Forest y modelos de ensamble con scikit-learn, aplicados al dataset Spaceship Titanic

Alumno:

Juan Pablo Pérez Gutiérrez – A01800483

Curso: Inteligencia artificial avanzada para la ciencia de datos

Profesor: Jorge Adolfo Ramírez Uresti

Grupo: 601

10 de Septiembre del 2026

1. Objetivo

Esta entrega corresponde a la segunda parte del Módulo 2. Mientras que en la primera parte el algoritmo se programó manualmente, aquí el objetivo es el opuesto: resolver el mismo problema apoyándose por completo en un framework de aprendizaje máquina y demostrar dominio sobre su uso y sobre la configuración del algoritmo.

El framework elegido es scikit-learn y el algoritmo principal es Random Forest, contrastado contra una Regresión Logística y un Gradient Boosting para justificar la elección con evidencia y no por preferencia. El problema es de clasificación binaria: predecir si un pasajero de la nave Spaceship Titanic fue transportado a otra dimensión tras la colisión de la nave con una anomalía espaciotemporal.

2. Dataset: con qué se entrenó y con qué se probó

Se utilizó el dataset Spaceship Titanic, obtenido de Kaggle. El archivo data/train.csv contiene 8,693 pasajeros descritos por 13 características más la variable objetivo Transported. La variable objetivo está balanceada (50.4 % transportados contra 49.6 % no transportados), lo cual hace que la exactitud sea una métrica interpretable.

Una precisión importante sobre los datos: **el archivo data/test.csv que incluye Kaggle no contiene la columna Transported**. Es el conjunto ciego con el que Kaggle califica su leaderboard, así que no sirve para medir desempeño. Por esa razón los tres subconjuntos se obtuvieron particionando data/train.csv de forma estratificada, conservando en cada parte la misma proporción de clases que el original.

Subconjunto	Muestras	% del total	% Transported	Uso
Entrenamiento	6,085	70 %	50.37 %	Ajuste y validación cruzada
Validación	1,304	15 %	50.31 %	Selección del modelo
Prueba	1,304	15 %	50.38 %	Evaluación final, una sola vez

La separación en tres subconjuntos y no en dos es deliberada. Si los hiperparámetros se eligieran mirando el conjunto de prueba, este dejaría de ser una medida imparcial, porque el modelo se estaría ajustando a él de forma indirecta. El conjunto de validación absorbe esa función y el de prueba permanece intacto hasta la evaluación final.

El archivo data/test.csv sí se utiliza, pero para lo único que sirve: demostrar predicción en lote sobre datos nuevos sin etiqueta (anexo A.2).

3. Uso del framework y configuración del algoritmo

El punto central de esta entrega no es que el modelo funcione, sino cómo se configuró el framework para que funcione bien. A continuación se describen las tres decisiones de diseño más relevantes.

3.1 Pipeline y ColumnTransformer

Todo el preprocesamiento vive dentro de un objeto Pipeline de scikit-learn en lugar de aplicarse por fuera sobre el DataFrame. Esto tiene dos consecuencias prácticas importantes.

- **Evita la fuga de datos.** Al entrar el pipeline completo a la validación cruzada, la imputación y el escalado se reajustan dentro de cada pliegue usando solo las filas de entrenamiento de ese pliegue. Si se imputara antes de partir los datos, la mediana calculada contendría información de las filas de prueba y las métricas quedarían infladas.
- **El modelo queda completo.** El objeto guardado recibe una fila tal como viene del CSV original y se encarga de derivar características, imputar, codificar y clasificar. Es imposible olvidar un paso al predecir datos nuevos.

El ColumnTransformer aplica un tratamiento distinto a cada grupo de columnas: a las numéricas, imputación por mediana y estandarización; a las categóricas, imputación por moda y codificación One-Hot con `handle_unknown="ignore"`, que impide que una categoría nunca vista en entrenamiento haga fallar la predicción.

```
Pipeline([
    ("caracteristicas", FunctionTransformer(derivar_caracteristicas)),
    ("preprocesamiento", ColumnTransformer([
        ("num", Pipeline([SimpleImputer(strategy="median"),
                          StandardScaler()])),
        ("cat", Pipeline([SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")])),
    ])),
    ("clasificador", RandomForestClassifier(...)),
])
```

La estandarización se activa solo para la Regresión Logística. Los modelos basados en árboles son invariantes a transformaciones monótonas de las variables, así que escalarlas no cambia nada; la Regresión Logística sí la necesita, porque su regularización penaliza los coeficientes y estos dependen de la escala de cada variable.

3.2 Ingeniería de características dentro del pipeline

Las características derivadas se crean con un FunctionTransformer que forma parte del pipeline, no con manipulaciones sueltas del DataFrame. De 13 columnas crudas se pasa a 16 características utilizables:

Nueva característica	Se obtiene de	Por qué
Deck, CabinNum, Side	Cabin (B/0/P)	La cabina son tres datos comprimidos en un texto; separados, el modelo puede usar cada uno
GroupSize	PassengerId (gggg_pp)	Los primeros dígitos identifican el grupo de viaje; su tamaño distingue a quien viajaba solo
TotalSpend	Suma de las 5 amenidades	Resume el consumo total en un solo número

NoSpend	TotalSpend == 0	Marca a quien no consumió nada, típicamente los pasajeros en criosueño
---------	-----------------	--

3.3 Búsqueda de hiperparámetros con GridSearchCV

Cada modelo se ajustó con GridSearchCV sobre el conjunto de entrenamiento, usando validación cruzada estratificada de 5 pliegues. En total se probaron 30 combinaciones de hiperparámetros, equivalentes a 150 ajustes de modelo.

Modelo	Hiperparámetros explorados	Comb.	Mejor configuración encontrada	Exactitud CV
Regresión Logística	$C \in \{0.01, 0.1, 1, 10\}$	4	$C = 0.1$	0.7916
Random Forest	$\text{max_depth} \in \{\text{None}, 12, 20\};$ $\text{min_samples_leaf} \in \{1, 2, 5\};$ $\text{max_features} \in \{\text{sqrt}, 0.5\}$	18	$\text{max_depth} = 20,$ $\text{min_samples_leaf} = 2,$ $\text{max_features} = \text{sqrt}$	0.8061
Gradient Boosting	$\text{learning_rate} \in \{0.05, 0.1\};$ $\text{max_iter} \in \{200, 400\};$ $\text{max_leaf_nodes} \in \{15, 31\}$	8	$\text{learning_rate} = 0.05, \text{max_iter} = 200,$ $\text{max_leaf_nodes} = 15$	0.8123

La elección de qué hiperparámetros explorar no es arbitraria: en el Random Forest, `max_depth` y `min_samples_leaf` controlan el sobreajuste de cada árbol, mientras que `max_features` controla qué tan decorrelacionados quedan los árboles entre sí, que es precisamente el mecanismo del que depende el ensamble para reducir la varianza. En el Gradient Boosting, `learning_rate` y `max_iter` se compensan entre sí, y la búsqueda confirmó esa relación: ganó la combinación de tasa baja (0.05) con el número menor de iteraciones (200) y árboles pequeños (15 hojas).

4. Comparación y selección del modelo

Con los tres modelos ya afinados, se comparó su desempeño sobre el conjunto de validación, que no participó en la búsqueda de hiperparámetros.

Modelo	Exactitud	F1	AUC
Regresión Logística	0.7929	0.7933	0.8823
Random Forest	0.8167	0.8117	0.9021
Gradient Boosting	0.8167	0.8185	0.9134

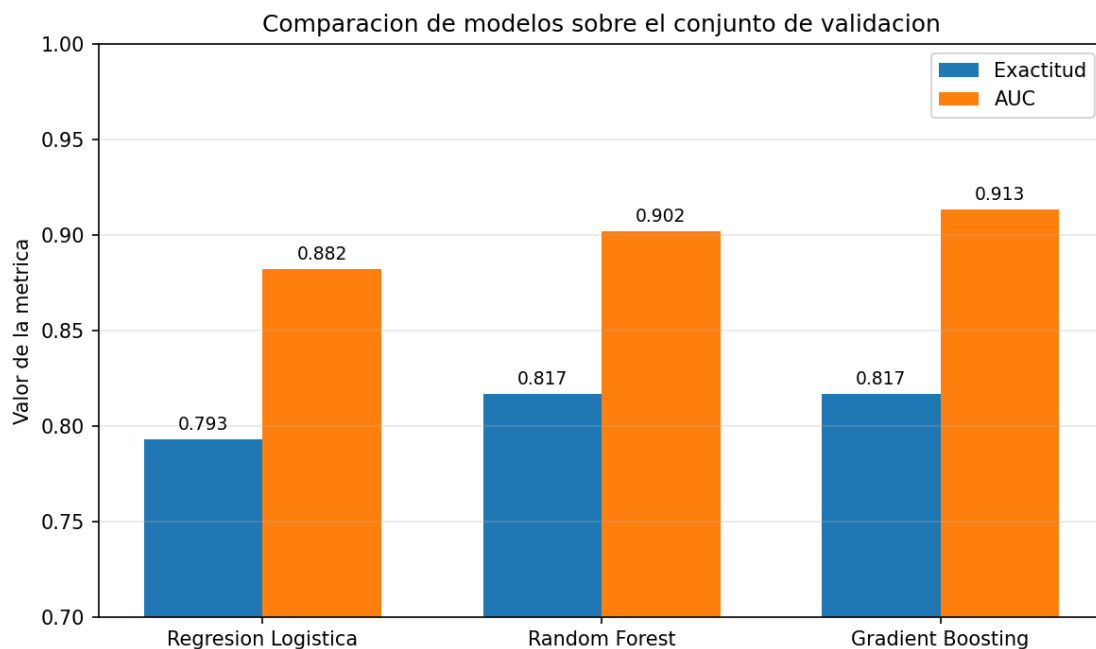


Figura 1. Exactitud y AUC de los tres modelos sobre el conjunto de validación.

Random Forest y Gradient Boosting empataron exactamente en exactitud (0.8167). Un empate así se resolvería, si no se especifica nada, por el orden en que los modelos aparecen en el código, lo cual es arbitrario. Por eso el criterio de desempate se fijó en el AUC, que es una medida más fina porque evalúa el ordenamiento completo de probabilidades y no solo las decisiones tomadas al umbral de 0.5. Con ese criterio el modelo seleccionado fue Gradient Boosting (AUC 0.9134 contra 0.9021).

Los dos modelos de ensamble superan a la Regresión Logística por más de dos puntos de exactitud, lo cual era esperable: la relación entre el gasto de un pasajero y su desenlace no es lineal, y un modelo lineal no puede capturar las interacciones entre variables que los árboles sí representan de forma natural.

5. Resultados sobre el conjunto de prueba

Todo lo anterior se decidió con entrenamiento y validación. El conjunto de prueba se evaluó una sola vez, aquí. La clase positiva es «el pasajero sí fue transportado».

5.1 Matrices de confusión

Modelo	VN	FP	FN	VP	Total
Regresión Logística	500	147	138	519	1,304
Random Forest	532	115	155	502	1,304
Gradient Boosting	514	133	124	533	1,304

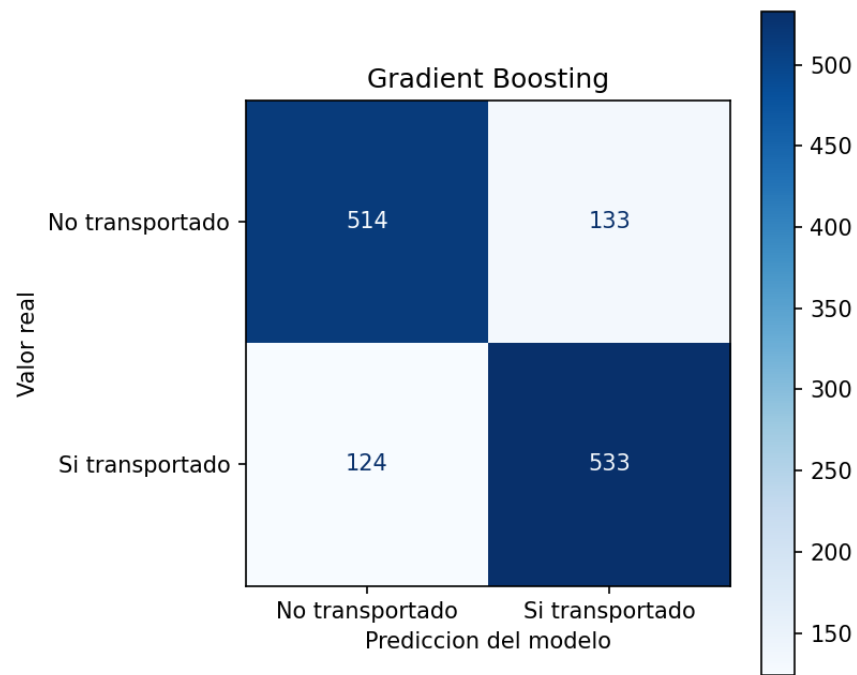


Figura 2. Matriz de confusión del modelo seleccionado (Gradient Boosting) sobre el conjunto de prueba.

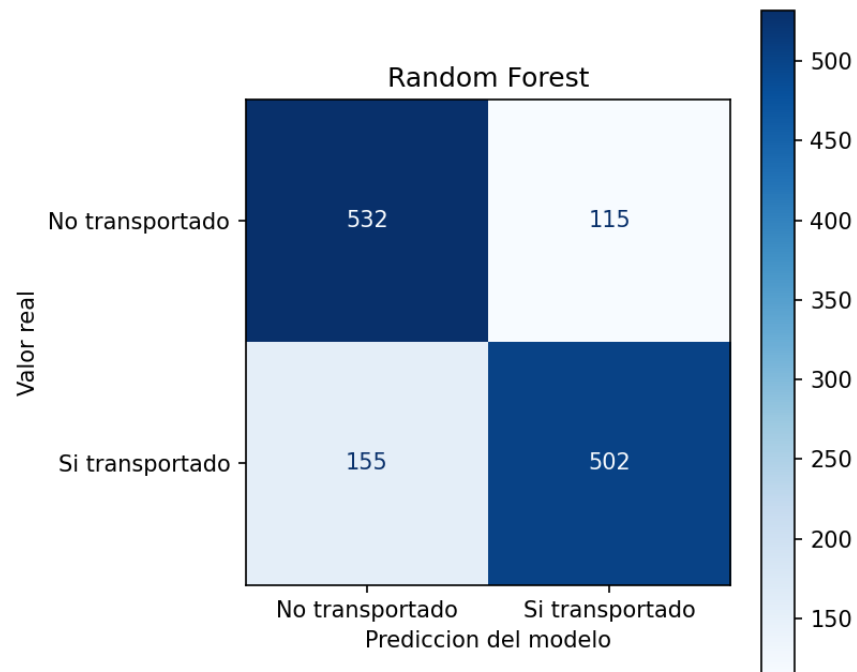


Figura 3. Matriz de confusión del Random Forest sobre el conjunto de prueba.

5.2 Métricas

Se reportan seis métricas porque cada una describe un aspecto distinto del comportamiento del modelo:

- **Exactitud:** es interpretable aquí precisamente porque las clases están balanceadas. La línea base de predecir siempre la clase mayoritaria da 0.5038, así que cualquier mejora sobre ese número es capacidad predictiva real.
- **Precisión y sensibilidad:** describen los dos tipos de error por separado. La exactitud sola puede esconder un modelo que acierta mucho en una clase y falla en la otra.
- **Especificidad:** completa el cuadro con el comportamiento en la clase negativa; es la métrica donde más se separan los modelos entre sí.
- **F1:** media armónica de precisión y sensibilidad; penaliza que una de las dos sea baja, cosa que un promedio simple no haría.
- **AUC:** evalúa la calidad del ordenamiento de probabilidades independientemente del umbral. Un modelo puede tener exactitud mediocre y aun así ordenar bien los casos.

Modelo	Exactitud	Precisión	Sensibilidad	Especificidad	F1	AUC
Línea base	0.5038	—	—	—	—	—
Regresión Logística	0.7814	0.7793	0.7900	0.7728	0.7846	0.8776
Random Forest	0.7929	0.8136	0.7641	0.8223	0.7881	0.8907
Gradient Boosting	0.8029	0.8003	0.8113	0.7944	0.8057	0.9022

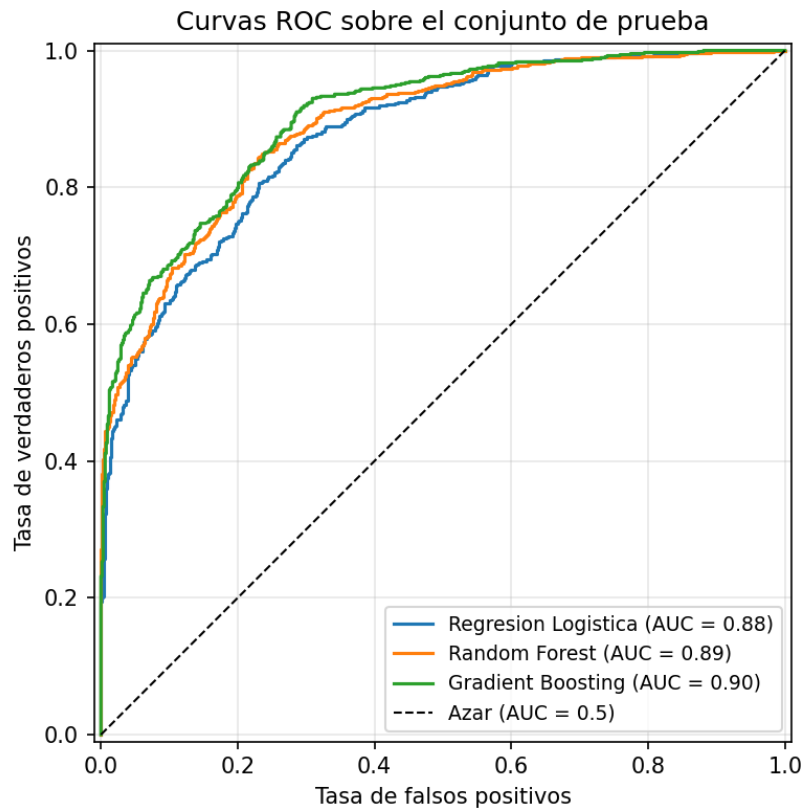


Figura 4. Curvas ROC de los tres modelos sobre el conjunto de prueba.

El modelo seleccionado alcanza 0.8029 de exactitud y 0.9022 de AUC sobre datos que nunca vio, contra una línea base de 0.5038. La ordenación de los tres modelos se mantuvo igual que en validación, lo cual indica que la selección no fue producto del azar de una partición particular.

6. Importancia de las características

Se usó importancia por permutación en lugar de la importancia por impureza que traen los árboles por defecto. La razón es que la permutación mide cuánto cae la exactitud del modelo al desordenar una variable sobre datos que el modelo no vio, mientras que la importancia por impureza se calcula sobre el entrenamiento y tiende a favorecer a las variables de alta cardinalidad. Además, al aplicarse sobre las columnas crudas de entrada, mide el aporte conjunto de todo lo que se deriva de cada una: la fila Cabin, por ejemplo, resume el efecto de Deck, CabinNum y Side juntos.

Característica	Caída de exactitud	Desv. est.
VRDeck	0.0553	± 0.0077
Spa	0.0492	± 0.0050
RoomService	0.0360	± 0.0045
FoodCourt	0.0341	± 0.0036
ShoppingMall	0.0176	± 0.0061

Cabin	0.0175	± 0.0055
CryoSleep	0.0110	± 0.0040
Age	0.0080	± 0.0041
PassengerId	0.0012	± 0.0022
HomePlanet	-0.0003	± 0.0045
Destination	-0.0003	± 0.0025
VIP	-0.0008	± 0.0003
Name	-0.0008	± 0.0000

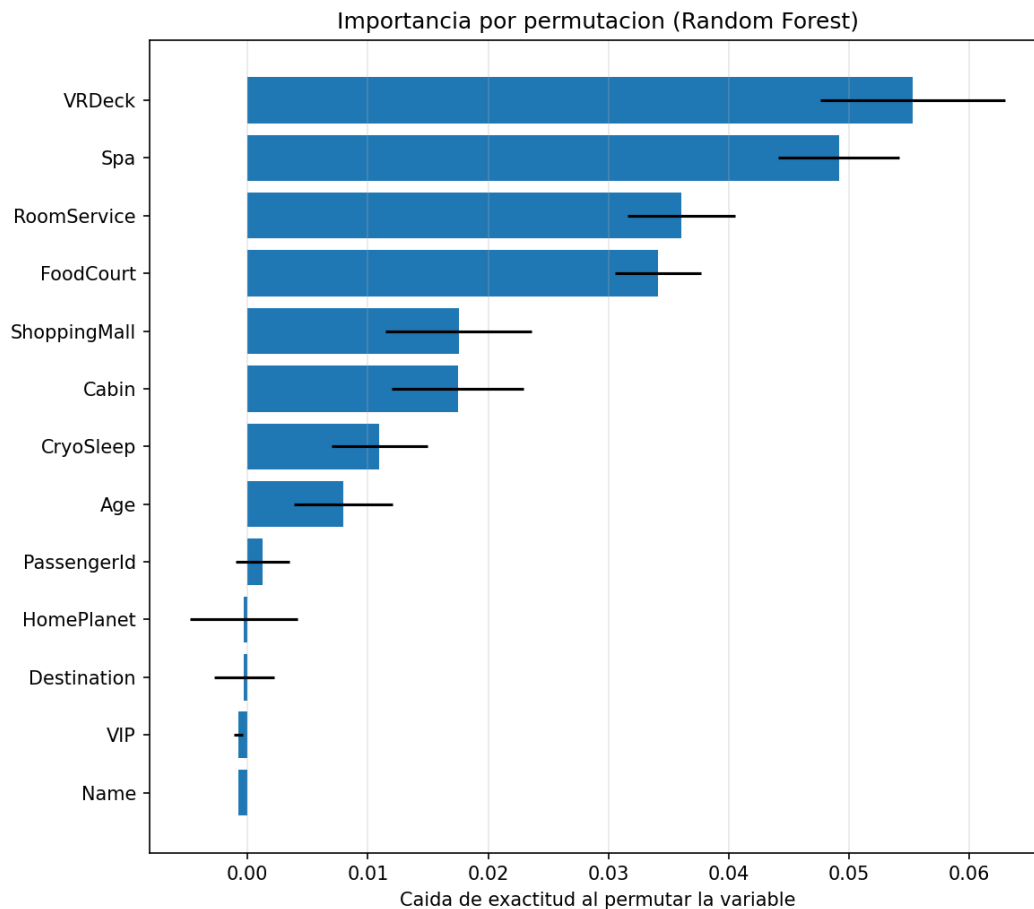


Figura 5. Importancia por permutación de cada característica de entrada.

Las cinco variables de gasto ocupan los primeros lugares y concentran la mayor parte de la señal: en qué gastó el pasajero importa más que quién era. Resulta llamativo que CryoSleep quede en séptimo lugar (0.0110) pese a ser muy predictiva por sí sola, y la explicación es la redundancia: un pasajero dormido no puede gastar, así que las variables de gasto ya capturan casi la misma información y permutar CryoSleep apenas afecta al modelo. Este es un efecto conocido de la importancia por permutación con variables correlacionadas.

En el otro extremo, HomePlanet, Destination, VIP y Name tienen importancia negativa o nula, es decir, permutarlas no empeora al modelo. En el caso de Name el valor es exactamente -0.0008 ± 0.0000 : una desviación estándar de cero a lo largo de las diez repeticiones confirma que el pipeline efectivamente descarta esa columna y nunca la usa.

7. Comparación contra la implementación manual

Como esta entrega resuelve el mismo problema que la Parte I, donde el algoritmo se programó desde cero, es posible comparar ambas implementaciones de forma directa. Para que la comparación sea válida, el Random Forest manual se entrenó con exactamente las mismas filas de entrenamiento y se evaluó sobre el mismo conjunto de prueba, con la misma configuración: 300 árboles, profundidad máxima 14 y mínimo 2 muestras por hoja. Los tiempos se midieron con esa misma configuración en ambos casos, de modo que la única diferencia entre las filas de la tabla sea la implementación.

Implementación	Exactitud	F1	AUC	Tiempo de entrenamiento
Manual (Parte I, código propio, 1 hilo)	0.7914	0.7979	0.8909	51.70 s
scikit-learn (1 hilo)	0.7914	0.7885	0.8911	3.35 s
scikit-learn (n_jobs = -1)	0.7914	0.7885	0.8911	1.32 s

El resultado es el más interesante del reporte. Con la misma configuración y los mismos datos, las dos implementaciones alcanzan exactamente la misma exactitud, 0.7914, coincidiendo hasta el cuarto decimal, y sus AUC difieren en 0.0002. Esto valida que la implementación manual de la Parte I era correcta: reproduce el comportamiento de una librería madura y ampliamente probada. La pequeña diferencia en F1 proviene de que ambas versiones resuelven de forma ligeramente distinta los empates al elegir el punto de corte de cada nodo, lo que reacomoda unos pocos casos alrededor del umbral de 0.5 sin cambiar la calidad global del ordenamiento.

Donde sí hay una diferencia enorme es en el tiempo. El código propio tarda 51.7 segundos en entrenar los 300 árboles; scikit-learn hace lo mismo en 3.35 segundos usando un solo hilo, y en 1.32 segundos aprovechando todos los núcleos. Son 15 veces más rápido en igualdad de condiciones y 39 veces con paralelismo. La ventaja no viene del algoritmo, que es el mismo, sino de la ingeniería: scikit-learn ejecuta sus rutinas críticas en Cython compilado y puede repartir el entrenamiento de los árboles entre varios núcleos mediante n_jobs, mientras que la versión propia corre en Python interpretado y en un solo hilo.

Esa es exactamente la lección de contrastar las dos entregas: programar el algoritmo a mano enseña por qué funciona, pero usar el framework es lo que permite iterar. Las 150 configuraciones de hiperparámetros que se probaron en la sección 3.3 habrían tomado horas con la implementación propia.

8. Análisis y conclusiones

- **El modelo aprende de verdad.** Contra una línea base de 0.5038, el modelo seleccionado alcanza 0.8029 de exactitud y 0.9022 de AUC sobre datos que nunca vio: treinta puntos por encima del azar.
- **La configuración importa tanto como el algoritmo.** La diferencia entre el mejor y el peor modelo afinado es de 2.2 puntos de exactitud, comparable a lo que aporta cambiar de familia de algoritmo. Explorar 150 ajustes con validación cruzada no fue un trámite: fue lo que permitió elegir con evidencia.
- **Los ensambles superan al modelo lineal.** Random Forest y Gradient Boosting le sacan más de dos puntos a la Regresión Logística, porque las relaciones entre el gasto del pasajero y su desenlace no son lineales ni aditivas.
- **El gasto es la señal dominante.** Las cinco variables de consumo ocupan los primeros lugares de importancia, mientras que el planeta de origen, el destino y el estatus VIP resultaron irrelevantes.
- **La implementación manual estaba bien hecha.** Con la misma configuración alcanza exactamente la misma exactitud que scikit-learn (0.7914), aunque tarda 15 veces más en entrenar en igualdad de condiciones y 39 veces más si se le permite a la librería usar todos los núcleos.

Limitaciones

- El techo de desempeño ronda el 80 % y no parece ser un problema de capacidad del modelo, sino de información disponible: hay pasajeros con perfiles prácticamente idénticos y desenlaces opuestos, que ningún modelo puede separar con estas características.
- No se explotó la estructura de grupos. Los pasajeros que viajaban juntos tienden a compartir desenlace, y el modelo solo usa el tamaño del grupo, no lo que ocurrió con los demás miembros.
- La selección del modelo se hizo sobre un conjunto de validación de 1,304 muestras, donde el error de medición ronda el ± 1.1 %. El empate entre Random Forest y Gradient Boosting refleja justamente eso: en la práctica los dos modelos son equivalentes dentro del margen de error.
- La búsqueda de hiperparámetros fue exhaustiva sobre rejillas discretas. Una búsqueda aleatoria o bayesiana exploraría el espacio de forma más eficiente si se quisieran probar más parámetros.

Anexo A. Ejecución del programa

La implementación está en el archivo `main_framework.py` y se ejecuta con un intérprete de Python normal; no depende de un IDE ni de un notebook.

```
python main_framework.py --entrenar           # ajusta, compara y evalua
python main_framework.py --predecir          # captura un pasajero por consola
python main_framework.py --predecir-csv data/test.csv # predicción en lote
```

A.1 Predicción individual desde consola

El modo interactivo pide los datos del pasajero uno por uno y devuelve la clase predicha junto con su probabilidad. Se probaron dos casos opuestos:

Caso 1 — pasajero de Europa, en criosueño, sin gasto:

```
SI fue transportado (probabilidad 98.8%)
```

Caso 2 — pasajero de la Tierra, despierto, con 5,450 de gasto total:

```
NO fue transportado (probabilidad 1.2%)
```

El modelo responde con alta confianza en ambos extremos y en la dirección que predicen los datos, lo cual confirma que las reglas aprendidas tienen sentido y no son un artefacto del ajuste.

A.2 Predicción en lote

Se predijeron las 4,277 filas del conjunto ciego de Kaggle: 2,193 pasajeros clasificados como transportados (51.3 %) y 2,084 como no transportados. El resultado, con la probabilidad de cada pasajero, se guarda en `resultados_framework/predicciones.csv`.

A.3 Reproducibilidad

Todas las semillas aleatorias están fijas en 42, tanto en las particiones como en los modelos, así que dos ejecuciones producen resultados idénticos. La bitácora completa de la corrida que generó los números de este reporte se guarda automáticamente en `resultados_framework/resultados.txt`, y las figuras en esa misma carpeta.